Christian DiPietrantonio
ME 5311: Computational Methods to Viscous Flows
April 4, 2026

Computer Assignment 02: Obtaining a Numerical Solution to the Zero Pressure Gradient Laminar Boundary Layer Equations

1. Description of the Problem

The objective of this project is to numerically solve the zero pressure gradient laminar boundary layer equations:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0, \qquad u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = \frac{\partial}{\partial y}\left(\nu\frac{\partial u}{\partial y}\right) \tag{1}$$

using the Crank-Nicolson algorithm to march in the x-direction, a stretched grid in the y-direction (where η is the stretched coordinate that is uniform in the computational domain), and a fourth-order central difference scheme for the discretization of η (except at the first interior points, where a second-order discretization is used). The following scales are considered: $\nu_\infty = 1.5 \times 10^{-6}$ m^2/s, $U_\infty = 30.0$ m/s, $L = 1.0$ m, $\delta = 0.005$ m. The initial velocity profile at $x = x_0$ is assumed to be:

$$\frac{u}{U_\infty} = \begin{cases} \frac{y}{\delta}\left(\frac{3}{2} - \frac{1}{2}\left(\frac{y}{\delta}\right)^2\right), & \text{for } \frac{y}{\delta} \leq 1 \\ 1, & \text{for } \frac{y}{\delta} > 1 \end{cases} \tag{2}$$

Finally, the domain uses $N = 150$ grid points in the y-direction with $y_{\max} = 10\delta$, and the solution is marched up to $40L$ downstream from x_0. Further objectives of the project are to: estimate the initial value of x based on the growth of the boundary layer, compare the results with the Blasius similarity solution, and calculate the displacement thickness, momentum thickness, and the shape factor in the y-direction.

2. Description of the Numerical Method

To non-dimensionalize the system, one can define $u^* = u/U_\infty$, $v^* = v/(U_\infty\delta/L)$, $x^* = x/L$, $y^* = y/\delta$, and $\nu^* = \nu/\nu_\infty$. Substituting these non-dimensional variables into the original boundary layer equations yields:

$$\frac{\partial u^*}{\partial x^*} + \frac{\partial v^*}{\partial y^*} = 0, \qquad u^*\frac{\partial u^*}{\partial x^*} + v^*\frac{\partial u^*}{\partial y^*} = C\frac{\partial}{\partial y^*}\left(\nu^*\frac{\partial u^*}{\partial y^*}\right) \tag{3}$$

Where $C = (1/Re_L)(L/\delta)^2 = 0.002$ and $Re_L = U_\infty L/\nu_\infty = 2 \times 10^7$. A sinh-based stretched grid clusters points near the wall. The physical coordinate y is related to the numerical domain coordinate η as follows:

$$y = \frac{y_{max}\sinh(s\eta)}{\sinh(s)} \tag{4}$$

Where $s = 10$ is a stretching factor and $\eta \in [0, 1]$. The Jacobian and its derivative can be defined as:

$$J^* = \frac{dy^*}{d\eta} = \frac{y_{\max}}{\delta} \cdot \frac{s\cosh(s\eta)}{\sinh(s)}, \qquad J^*_\eta = \frac{dJ^*}{d\eta} = \frac{y_{\max}}{\delta} \cdot \frac{s^2\sinh(s\eta)}{\sinh(s)} \tag{5}$$

Using the chain rule, the derivatives with respect to y^* can now be expressed as derivatives

with respect to η:

$$\frac{\partial}{\partial y^*} = \frac{1}{J^*}\frac{\partial}{\partial \eta}, \qquad \frac{\partial^2}{\partial y^{*2}} = \frac{1}{J^{*2}}\frac{\partial^2}{\partial \eta^2} - \frac{J^*_\eta}{J^{*3}}\frac{\partial}{\partial \eta} \tag{6}$$

Therefore, expanding $\frac{\partial}{\partial y^*}\left(\nu^* \frac{\partial u^*}{\partial y^*}\right)$ in terms of η and substituting the definitions above into the non-dimensionalized system yields the final non-dimensional system of equations in terms of η:

$$\frac{\partial v^*}{\partial \eta} = -J^* \frac{\partial u^*}{\partial x^*}, \qquad u^* \frac{\partial u^*}{\partial x^*} + A_j \frac{\partial u^*}{\partial \eta} + B_j \frac{\partial^2 u^*}{\partial \eta^2} = 0 \tag{7}$$

where the coefficients A_j and B_j are defined as:

$$A_j = \frac{v^*_j}{J^*_j} - \frac{C}{J^{*2}_j}\frac{\partial \nu^*}{\partial \eta}\bigg|_j + \frac{C\,\nu^*_j\,J^*_{\eta,j}}{J^{*3}_j}, \qquad B_j = \frac{-C\,\nu^*_j}{J^{*2}_j} \tag{8}$$

It is important to note that for laminar flow ($\nu^* = 1$, $\partial \nu^*/\partial \eta = 0$), so A_j and B_j reduce to $A_j = v^*_j/J^*_j + C\,J^*_{\eta,j}/J^{*3}_j$ and $B_j = -C/J^{*2}_j$. To discretize the continuity equation, the following 4th-order approximation for $\frac{\partial v^*}{\partial \eta}$ and 2nd-order approximation for $\frac{\partial u^*}{\partial x^*}$ are considered for the interior domain ($j = 2$ to $j = N-3$):

$$\left(\frac{\partial u^*}{\partial x^*}\right)^{i+1}_{j-\frac{1}{2}} = 0.5\left(\frac{3u^{*,i+1}_j - 4u^{*,i}_j + u^{*,i-1}_j}{2\Delta x^*} + \frac{3u^{*,i+1}_{j-1} - 4u^{*,i}_{j-1} + u^{*,i-1}_{j-1}}{2\Delta x^*}\right) \tag{9}$$

$$\left(\frac{\partial v^*}{\partial \eta}\right)^{i+1}_{j-\frac{1}{2}} = \frac{v^{*,i+1}_{j-2} - 27v^{*,i+1}_{j-1} + 27v^{*,i+1}_j - v^{*,i+1}_{j+1}}{24\Delta\eta} \tag{10}$$

Substituting these approximations into the continuity equation and rearranging yields:

$$\begin{aligned} &\frac{1}{24\Delta\eta}v^{*,i+1}_{j-2} - \frac{27}{24\Delta\eta}v^{*,i+1}_{j-1} + \frac{27}{24\Delta\eta}v^{*,i+1}_j - \frac{1}{24\Delta\eta}v^{*,i+1}_{j+1} = \\ &-(J^*)^{i+1}_{j-\frac{1}{2}}\left[\frac{0.5}{2\Delta x^*}\left(3u^{*,i+1}_j - 4u^{*,i}_j + u^{*,i-1}_j + 3u^{*,i+1}_{j-1} - 4u^{*,i}_{j-1} + u^{*,i-1}_{j-1}\right)\right] \end{aligned} \tag{11}$$

Near the boundary and at the initial step in x ($i = 1$, $j = 1$ and $j = N-2$), a 4th-order approximation is used for $\frac{\partial v^*}{\partial \eta}$ and 1st-order approximation is used for $\frac{\partial u^*}{\partial x^*}$:

$$\begin{aligned} &\frac{1}{24\Delta\eta}v^{*,i+1}_{j-2} - \frac{27}{24\Delta\eta}v^{*,i+1}_{j-1} + \frac{27}{24\Delta\eta}v^{*,i+1}_j - \frac{1}{24\Delta\eta}v^{*,i+1}_{j+1} = \\ &-(J^*)^{i+1}_{j-\frac{1}{2}}\left[\frac{0.5}{\Delta x^*}\left(u^{*,i+1}_j - u^{*,i}_j + u^{*,i+1}_{j-1} - u^{*,i}_{j-1}\right)\right] \end{aligned} \tag{12}$$

Applying the Crank-Nicolson scheme and lagging the non-linear term $u^*\frac{\partial u^*}{\partial x^*}$ yields the following expression for the momentum equation:

$$u^{*,i}_j\left(\frac{u^{*,i+1}_j - u^{*,i}_j}{\Delta x^*}\right) + \frac{A_j}{2}\left[\left(\frac{\partial u^*}{\partial \eta}\right)_{i,j} + \left(\frac{\partial u^*}{\partial \eta}\right)_{i+1,j}\right] + \frac{B_j}{2}\left[\left(\frac{\partial^2 u^*}{\partial \eta^2}\right)_{i,j} + \left(\frac{\partial^2 u^*}{\partial \eta^2}\right)_{i+1,j}\right] = 0 \tag{13}$$

The following 4th-order approximations are then considered for the interior domain:

$$\left(\frac{\partial u^*}{\partial \eta}\right)_{i,j} = \frac{u^{*,i}_{j-2} - 8u^{*,i}_{j-1} + 8u^{*,i}_{j+1} - u^{*,i}_{j+2}}{12\Delta\eta} \tag{14}$$

$$\left(\frac{\partial^2 u^*}{\partial \eta^2}\right)_{i,j} = \frac{-u^{*,i}_{j-2} + 16u^{*,i}_{j-1} - 30u^{*,i}_{j} + 16u^{*,i}_{j+1} - u^{*,i}_{j+2}}{12\Delta\eta^2} \tag{15}$$

Substituting these approximations into Equation 13 and rearranging yields:

$$\begin{aligned}&(\phi_1 - \phi_2)u^{*,i+1}_{j-2} + (-8\phi_1 + 16\phi_2)u^{*,i+1}_{j-1} + (u^{*,i}_j/\Delta x^* - 30\phi_2)u^{*,i+1}_j \\ &+ (8\phi_1 + 16\phi_2)u^{*,i+1}_{j+1} + (-\phi_1 - \phi_2)u^{*,i+1}_{j+2} = (-\phi_1 + \phi_2)u^{*,i}_{j-2} + (8\phi_1 - 16\phi_2)u^{*,i}_{j-1} \\ &+ (u^{*,i}_j/\Delta x^* + 30\phi_2)u^{*,i}_j + (-8\phi_1 - 16\phi_2)u^{*,i}_{j+1} + (\phi_1 + \phi_2)u^{*,i}_{j+2}\end{aligned} \tag{16}$$

Where $\phi_1 = A_j/(24\Delta\eta)$ and $\phi_2 = B_j/(24\Delta\eta^2)$. Near the wall, the following 2nd-order approximations are used:

$$\left(\frac{\partial u^*}{\partial \eta}\right)_{i,j} = \frac{u^{*,i}_{j+1} - u^{*,i}_{j-1}}{2\Delta\eta} \tag{17}$$

$$\left(\frac{\partial^2 u^*}{\partial \eta^2}\right)_{i,j} = \frac{u^{*,i}_{j+1} - 2u^{*,i}_{j} + u^{*,i}_{j-1}}{\Delta\eta^2} \tag{18}$$

Substituting these approximations into Equation 13 and rearranging yields:

$$\begin{aligned}&(-\psi_1 + \psi_2)u^{*,i+1}_{j-1} + (u^{*,i}_j/\Delta x^* - 2\psi_2)u^{*,i+1}_j + (\psi_1 + \psi_2)u^{*,i+1}_{j+1} = \\ &(\psi_1 - \psi_2)u^{*,i}_{j-1} + (u^{*,i}_j/\Delta x^* + 2\psi_2)u^{*,i}_j + (-\psi_1 - \psi_2)u^{*,i}_{j+1}\end{aligned} \tag{19}$$

Where $\psi_1 = A_j/(4\Delta\eta)$ and $\psi_2 = B_j/(2\Delta\eta^2)$. The boundary conditions are known to be $u^*_0 = 0$ at the wall and $u^*_{N-1} = 1$ at the freestream. Therefore for the 2nd-order stencil, at $j = 1$, the $u^{*,i,i+1}_{j-1}$ terms vanish and at $j = N-2$, the $u^{*,i,i+1}_{j+1}$ terms become $(\psi_1 + \psi_2)$ and $(-\psi_1 - \psi_2)$. The former of the two is moved to the right-hand side, and they collectively become $-2(\psi_1 + \psi_2)$. For the 4th-order stencil, at $j = 2$, the $u^{*,i,i+1}_{j-2}$ terms vanish and at $j = N-3$, the $u^{*,i,i+1}_{j+2}$ terms become $(-\phi_1 - \phi_2)$ and $(\phi_1 + \phi_2)$. Again, the former of the two is moved to the right-hand side, and they collectively become $2(\phi_1 + \phi_2)$. The numerical solver shown in Appendix A can now be programmed to solve the discretized system above and complete the aforementioned objectives of the project.

3. Presentation of Results

The initial value of x can be estimated from the Blasius relation for boundary layer thickness $\delta = c\sqrt{x + x_0}$, where $c = 4.91\sqrt{\nu_\infty/U_\infty}$. Assuming that the leading edge of the wall/plate is located at $x = 0$, the initial value of x can be expressed as $x_0 = \delta^2/c^2$. Therefore:

$$x_0 = \frac{0.005^2}{\left(4.91\sqrt{\frac{1.5\times10^{-6}}{30}}\right)^2} \approx 20.74 \text{ m} \tag{20}$$

This results in $Re_{x_0} \approx 4.15 \times 10^8$. Furthermore, since $x^* = x/L$, the non-dimensionalized initial value of x is found as $x^*_0 = 20.74$. The solution is then marched $40L$ downstream from

x_0^* to $x_{max}^* = 60.74$.

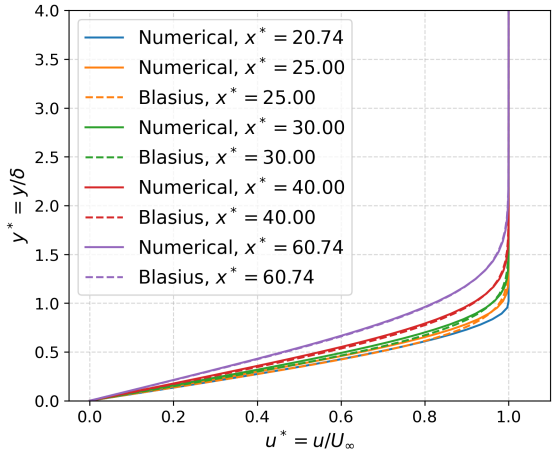

Figure 1: Velocity profiles u^* vs. y/δ and Blasius solutions at four downstream locations.

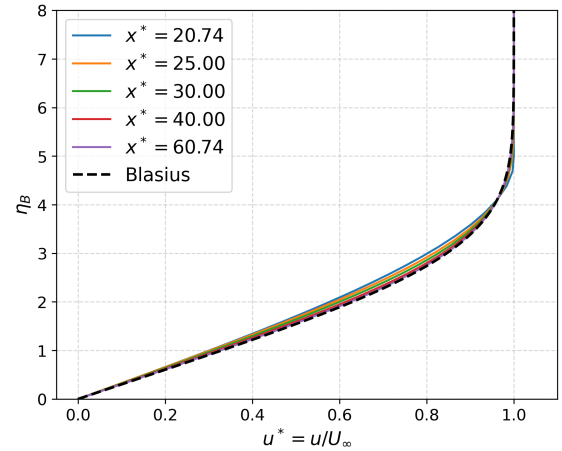

Figure 2: Similarity profiles: $\eta_B = y\sqrt{U_\infty/(\nu x)}$ vs. u^* at four downstream locations.

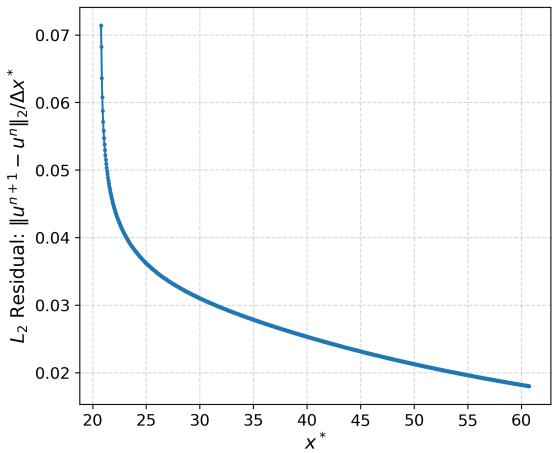

Figure 3: L_2 residual (normalized by Δx^*) vs. x^*.

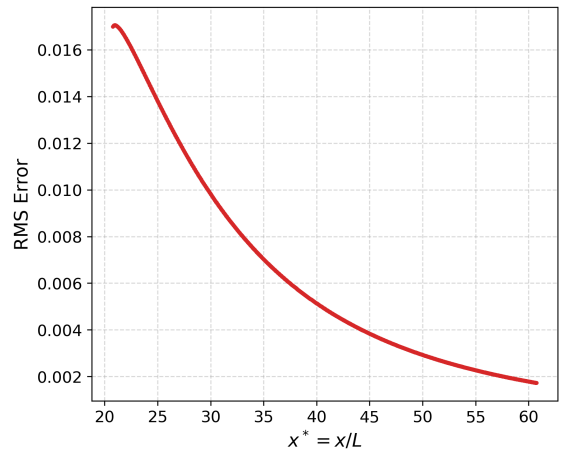

Figure 4: RMS error vs. x^*.

In terms of η, the non-dimensional displacement thickness can be expressed as:

$$\frac{\delta^*}{\delta} = \int_0^1 (1 - u^*)\, J^* d\eta \tag{21}$$

the non-dimensional momentum thickness can be expressed as:

$$\frac{\theta}{\delta} = \int_0^1 u^* (1 - u^*)\, J^* d\eta \tag{22}$$

and the shape factor as:

$$H = \frac{\delta^*}{\theta} \tag{23}$$

For the Blasius solution, the displacement thickness, momentum thickness, and shape factor are known to be:

$$\delta^*_{\text{Blas}} = 1.72\sqrt{\frac{\nu x}{U_\infty}}, \qquad \theta_{\text{Blas}} = 0.664\sqrt{\frac{\nu x}{U_\infty}}, \qquad H_{\text{Blas}} = \frac{1.72}{0.664} \approx 2.59 \tag{24}$$

Table 1: Displacement thickness, momentum thickness, and shape factor at each output location, compared with Blasius values.

x^*	δ^*/δ	θ/δ	H	$\delta^*_{\text{Blas}}/\delta$	$\theta_{\text{Blas}}/\delta$	H_{Blas}
20.74	0.3750	0.1393	2.692	0.3503	0.1352	2.59
25.00	0.4063	0.1514	2.684	0.3846	0.1485	2.59
30.00	0.4378	0.1645	2.661	0.4213	0.1627	2.59
40.00	0.4961	0.1886	2.631	0.4865	0.1878	2.59
60.74	0.6031	0.2313	2.608	0.5995	0.2314	2.59

4. Discussion of Results

4.1. General description

The numerical solution exhibits excellent agreement with the Blasius similarity solution at the downstream locations. Figure 1 shows that the velocity profiles at $x^* = 25$, 30, 40, and 60.74 closely match the corresponding Blasius profiles. From Figure 1, it can also be observed that the boundary layer grows as expected, and the velocity gradient at the wall decreases as x^* increases. The similarity profiles in Figure 2 highlight a key feature of a laminar boundary layer on a flat plate and self-similar solutions. When the velocity profiles at different x^* locations are plotted against the Blasius similarity variable $\eta_B = y\sqrt{U_\infty/(\nu x)}$, they collapse onto a single curve (the Blasius solution). One can observe that the initial profile at $x^* = 20.74$ deviates the most from the Blasius solution. This is because the cubic approximation profile is not a solution to the governing equations, but rather an initial guess. However, subsequent velocity profiles converge to the Blasius solution as x^* increases, and by $x^* = 60.74$, the numerical and Blasius solutions are nearly the same. It is important to note that accuracy improves further downstream in x^* because the momentum equation is parabolic, therefore information is only propagated in the downstream direction and the impact of the initial disturbance (cubic profile) decreases as the solution marches forward. Table 1 also demonstrates that the displacement thickness, momentum thickness, and shape factor all approach their corresponding Blasius values as x^* increases. From the data, we can observe that the initial cubic profile overestimates the displacement thickness, momentum thickness, and shape factor compared to the Blasius solution. Nevertheless, by $x^* = 60.74$, the numerical δ^*/δ, θ/δ, and H are within 0.6%, 0.04%, and 0.7% of their respective Blasius values.

4.2. Accuracy and stability

The L2 residual shown in Figure 3 decreases as x^* increases, which indicates convergence of the solution. The initial residual is largest near $x^* = 20.74$ because the initial cubic condition must undergo rapid changes to satisfy the governing equations. The Root Mean Square (RMS) error between the numerical and Blasius solutions shown in Figure 4 also decreases (from a maximum of roughly 0.017 near $x^* = 20.74$ due to the initial cubic profile) as x^* increases. Nevertheless, Figure 4 indicates high accuracy of the results over the entire computational domain. The accuracy of the solution is further confirmed by the differences in displacement thickness, momentum thickness, and shape factor listed above. Finally, given that neither the solutions nor the L2 residual show any signs of oscillations or unphysical behavior, the solution is stable. However, this is to be expected because the Crank-Nicolson scheme is unconditionally stable.

A. Copy of Program Listing

```python
# Christian DiPietrantonio
# ME 5311: Computational Methods to Viscous Flows
# Computer Assignment 02
# 04/04/2026

import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import solve_banded

# ------------------------------------------------------------------
# 1. SET PARAMETERS
# ------------------------------------------------------------------

def set_parameters():
    """Define all physical constants, grid parameters, and nondimensional quantities."""

    params = {}
    params["nu_inf"] = 1.5e-6
    params["U_inf"] = 30.0
    params["L"] = 1.0
    params["delta"] = 0.005
    params["N"] = 150  # number of grid points in eta (y-direction)
    params["y_max"] = 10 * params["delta"] # final y
    params["s"] = 10   # stretching factor
    params["Re_L"] = params["U_inf"] * params["L"] / params["nu_inf"]
    params["C"] = (1.0 / params["Re_L"]) * (params["L"] / params["delta"])**2 # non-dimensional coefficient
    params["x0"] = (params["U_inf"] * params["delta"]**2) / (params["nu_inf"] * 4.91**2) # initial x from Blasius relation
    params["x0_star"] = params["x0"] / params["L"] # non-dimensional initial x, x0*
    params["x_max"] = params["x0"] + 40 * params["L"] # final x
    params["x_max_star"] = params["x_max"] / params["L"] # non-dimensional final x, x_max*
    params["Nx"] = 10000   # number of steps in x*
    params["dx_star"] = (params["x_max_star"] - params["x0_star"]) / params["Nx"] # step size in x*
    params["deta"] = 1.0 / (params["N"] - 1) # step size in eta
    params["x_output"] = [params["x0_star"], 25, 30, 40, params["x_max_star"]]

    return params

# ------------------------------------------------------------------
# 2. COMPUTE GRID
# ------------------------------------------------------------------

def compute_grid(params):
    """Compute the stretched grid, the Jacobian and its derivative."""

    N         = params["N"]
    y_max     = params["y_max"]
    s         = params["s"]
    delta     = params["delta"]
```

```python
    eta       = np.linspace(0.0, 1.0, N)
    y_phys    = y_max * np.sinh(s * eta) / np.sinh(s)
    y_star    = y_phys / delta
    J_star    = (y_max * s * np.cosh(s * eta)) / (delta * np.sinh(s))
    J_star_eta = (y_max * s**2 * np.sinh(s * eta)) / (delta * np.sinh(s))

    return eta, y_phys, y_star, J_star, J_star_eta

# ------------------------------------------------------------------
# 3. COMPUTE INITIAL CONDITION
# ------------------------------------------------------------------

def compute_initial_condition(params, y_star):
    """Compute the initial velocity profile at x0."""

    N = params["N"]
    u_star_0 = np.ones(N)

    for j in range(N):
        y_j = y_star[j]

        if y_j <= 1.0:
            u_star_0[j] = y_j * (1.5 - 0.5 * y_j**2)
        else:
            u_star_0[j] = 1.0

    return u_star_0

# ------------------------------------------------------------------
# 4.  COMPUTE nu* and dnu*/deta
# ------------------------------------------------------------------

def compute_nu(params, u_star, J_star):
    """Compute the non-dimensional kinematic viscosity nu* and its derivative with
        respect to eta."""

    N = params["N"]
    nu_star = np.ones(N)
    dnu_star_deta = np.zeros(N)

    return nu_star, dnu_star_deta

# ------------------------------------------------------------------
# 5.  COMPUTE COEFFICIENTS A_j and B_j
# ------------------------------------------------------------------

def compute_coefficients(params, v_star, J_star, J_star_eta, nu_star, dnu_star_deta):
    """Compute the coefficients A_j and B_j for the discretized system."""

    C = params["C"]
    A_j = (v_star / J_star) - ((C / J_star**2) * dnu_star_deta) + ((C * nu_star *
        J_star_eta) / J_star**3)
    B_j = -C * nu_star / J_star**2

```

```python
    return A_j, B_j

# ------------------------------------------------------------------
# 6.  COMPUTE v* FROM CONTINUITY
# ------------------------------------------------------------------

def compute_v_star(u_star_new, u_star_old, u_star_prev, params, J_star):
    """Compute v* from the transformed/non-dimensional continuity equation
       using a 4th-order approximation for dv*/deta and a 2nd-order approximation
       for du*/dx* for the interior domain (j = 2 to N-3), and a 4th-order approximation
       for dv*/deta and a 1st-order approximation for du*/dx* near the boundary/at
       the initial step in x (i = 1, j = 1 and j = N-2)."""

    N         = params["N"]
    dx_star   = params["dx_star"]
    deta      = params["deta"]
    l_diag        = 2 # number of lower diagonals in banded system
    u_diag        = 1 # number of upper diagonals in banded system

    # ---------- Compute du*/dx* at each grid point ---------------
    if u_star_prev is None:
        du_dx_star = (u_star_new - u_star_old) / dx_star
    else:
        du_dx_star = (3.0 * u_star_new - 4.0 * u_star_old + u_star_prev) / (2.0 * dx_star)

    # ---------- Compute midpoint RHS values -----------------------
    f_mid = np.zeros(N)
    for j in range(1, N):
        J_mid             = 0.5 * (J_star[j] + J_star[j-1])
        du_dx_star_mid    = 0.5 * (du_dx_star[j] + du_dx_star[j-1])
        f_mid[j]          = -J_mid * du_dx_star_mid

    # ---------- Set up banded system ------------------------------
    # Number of unknowns = N-1, v*_0 = 0 (Wall BC)
    # Unknowns: x[k] = v*_{k+1} for k = 0 to N-2
    n = N - 1 # number of unknowns (v_star[1] to v_star[N-1])
    c = 1.0 / (24.0 * deta) # stencil coefficient

    # Banded matrix (l + u + 1, n) where l = 2 lower diagonals, u = 1 upper diagonal
    # element[row, col] maps to ab[u + row - col, col]
    ab = np.zeros((l_diag + u_diag + 1, n))
    rhs_v_star = np.zeros(n)

    # Row 0: Midpoint Rule for v*_1
    ab[u_diag, 0] = 1.0 / deta  # [row=0, col=0] -> ab[1 + 0 - 0, 0] = ab[1, 0]
    rhs_v_star[0] = f_mid[1]

    # Rows 1 to N-3 (corresponding to v*_2 to v*_{N-2}): 4th order stencil
    for j in range(2, N-1):
        r = j - 1                           # row index in banded system

        if j - 2 >= 1:
            ab[u_diag + r - (r-2), r-2] = c  # map value two columns behind main diagonal
                (2nd lower diagonal)
```

```
                                            # ab[3, r-2]: v*_{j-2}
        # if j = 2, v*_0 = 0, so no contribution

        ab[u_diag + r - (r-1), r-1] = -27 * c # map value one column behind main diagonal (1st lower diagonal)
                                            # ab[2, r-1]: v*_{j-1}

        ab[u_diag, r]             = 27 * c    # map value on main diagonal
                                            # ab[1, r]: v*_j

        ab[u_diag + r - (r+1), r+1] = -c     # map value one column ahead of main diagonal (1st upper diagonal)
                                            # ab[0, r+1]: v*_{j+1}

        rhs_v_star[r] = f_mid[j]

    # Row N-2: Midpoint Rule for v*_{N-1}
    r = N - 2
    ab[u_diag + r - (r-1), r-1] = -1.0 / deta # map value one column behind main diagonal (1st lower diagonal)
                                            # ab[2, r-1]: v*_{N-2}

    ab[u_diag + r - r, r]     = 1.0 / deta   # map value on main diagonal
                                            # ab[1, r]: v*_{N-1}

    rhs_v_star[r] = f_mid[N-1]

    # ---------- Solve banded system ------------------------------
    x = solve_banded((l_diag, u_diag), ab, rhs_v_star)

    # ---------- Assemble full v_star array ------------------------
    v_star = np.zeros(N)
    v_star[0] = 0.0        # wall BC
    v_star[1:] = x         # interior and freestream points

    return v_star

# ------------------------------------------------------------------
# 7.  BUILD LINEAR SYSTEM FOR MOMENTUM EQUATION
# ------------------------------------------------------------------

def build_system(params, u_old, A_j, B_j):
    """Build the banded matrix and RHS vector for the linear system resulting from the discretization
       of the momentum equation."""

    N         = params["N"]
    dx_star   = params["dx_star"]
    deta      = params["deta"]
    l_diag    = 2 # number of lower diagonals in banded system
    u_diag    = 2 # number of upper diagonals in banded system

    # Number of unknowns = N-2, u*_0 = 0 (Wall BC), u*_{N-1} = 1 (Freestream BC)
    # Unknowns: x[k] = u*_{k+1} for k = 0 to N-3
```

```python
    n         = N - 2 # number of unknowns (u_star[1] to u_star[N-2])

    # Banded matrix (l + u + 1, n) where l = 2 lower diagonals, u = 2 upper diagonals
    # element[row, col] maps to ab[u + row - col, col]
    ab = np.zeros((l_diag + u_diag + 1, n))
    rhs = np.zeros(n)

    # Row 0: j=1 (near wall, 2nd order)
    j = 1; k = 0
    psi_1        = A_j[j] / (4.0 * deta)
    psi_2        = B_j[j] / (2.0 * deta**2)
    ab[2, k]     = (u_old[j] / dx_star) - 2 * psi_2 # [row=0, col=0] -> ab[2 + 0 - 0, 0] = ab[2, 0]
    ab[1, k + 1] = psi_1 + psi_2                    # [row=0, col=1] -> ab[2 + 0 - 1, 1] = ab[1, 1]
    rhs[k]       = ((u_old[j]**2 / dx_star) + 2.0 * psi_2 * u_old[j]) + (-psi_1 - psi_2) * u_old[j+1]

    # Rows 1 to N-4 (corresponding to u*_2 to u*_{N-3}): 4th order stencil
    for j in range(2, N-2):
        k = j - 1  # row index in banded system
        phi_1    = A_j[j] / (24.0 * deta)
        phi_2    = B_j[j] / (24.0 * deta**2)
        c_jm2    = phi_1 - phi_2                        # coefficient for u^{*,i+1}_{j-2}
        c_jm1    = -8.0 * phi_1 + 16.0 * phi_2          # coefficient for u^{*,i+1}_{j-1}
        c_j      = u_old[j] / dx_star - 30.0 * phi_2    # coefficient for u^{*,i+1}_j
        c_jp1    = 8.0 * phi_1 + 16.0 * phi_2           # coefficient for u^{*,i+1}_{j+1}
        c_jp2    = -phi_1 - phi_2                       # coefficient for u^{*,i+1}_{j+2}
        r_jm2    = -phi_1 + phi_2                       # coefficient for u^{*,i}_{j-2}
        r_jm1    = 8.0 * phi_1 - 16.0 * phi_2           # coefficient for u^{*,i}_{j-1}
        r_j      = u_old[j] / dx_star + 30.0 * phi_2    # coefficient for u^{*,i}_j
        r_jp1    = -8.0 * phi_1 - 16.0 * phi_2          # coefficient for u^{*,i}_{j+1}
        r_jp2    = phi_1 + phi_2                        # coefficient for u^{*,i}_{j+2}

        if j - 2 >= 1:
            ab[u_diag + k - (k-2), k-2]   = c_jm2 # map value two columns behind main diagonal (2nd lower diagonal)
                                                  # ab[4, k-2]: u*_{j-2}

        ab[u_diag + k - (k-1), k-1]      = c_jm1 # map value one column behind main diagonal (1st lower diagonal)
                                                  # ab[3, k-1]: u*_{j-1}

        ab[u_diag, k]                    = c_j   # map value on main diagonal
                                                  # ab[2, k]: u*_j

        ab[u_diag + k - (k+1), k+1]      = c_jp1 # map value one column ahead of main diagonal (1st upper diagonal)
                                                  # ab[1, k+1]: u*_{j+1}

        if j + 2 <= N - 2:
            ab[u_diag + k - (k+2), k+2]   = c_jp2 # map value two columns ahead of main diagonal (2nd upper diagonal)
                                                  # ab[0, k+2]: u*_{j+2}
```

```python

        rhs_val = r_j * u_old[j]

        if j - 2 >= 1:
            rhs_val += r_jm2 * u_old[j-2]    # contribution from u^{*,i}_{j-2}
        rhs_val += r_jm1 * u_old[j-1]        # contribution from u^{*,i}_{j-1}
        rhs_val += r_jp1 * u_old[j+1]        # contribution from u^{*,i}_{j+1}
        if j + 2 <= N - 2:
            rhs_val += r_jp2 * u_old[j+2]    # contribution from u^{*,i}_{j+2}
        else:
            rhs_val += r_jp2 * 1.0           # contribution from u^{*,i}_{N-1} = 1.0 (Freestream BC)
            rhs_val -= c_jp2 * 1.0           # contribution from u^{*,i+1}_{N-1} = 1.0 (Freestream BC)
        rhs[k] = rhs_val

    # Row N-3: j=N-2 (near freestream, 2nd order)
    j = N - 2; k = j - 1
    psi_1              = A_j[j] / (4.0 * deta)
    psi_2              = B_j[j] / (2.0 * deta**2)
    ab[3, k-1] = -psi_1 + psi_2                  # [row=N-3, col=N-4] -> ab[2 + N-3 - N-4, N-4] = ab[3, N-4]
    ab[2, k]  = u_old[j] / dx_star - 2.0 * psi_2  # [row=N-3, col=N-3] -> ab[2 + N-3 - N-3, N-3] = ab[2, N-3]
    # include contributions from u^{*,i+1}_{N-1} and u^{*,i}_{N-1} = 1.0 (Freestream BC) in RHS
    rhs[k]    = (psi_1 - psi_2) * u_old[j-1] + (u_old[j]**2 / dx_star) + (2.0 * psi_2 * u_old[j]) + (-psi_1 - psi_2) * 1.0 - (psi_1 + psi_2) * 1.0

    return ab, rhs, l_diag, u_diag

# ------------------------------------------------------------------
# 8.  STEP IN X*
# ------------------------------------------------------------------

def step_in_x(params, u_star_0, J_star, J_star_eta):
    """March from x0_star to x_max_star"""

    N        = params["N"]
    Nx       = params["Nx"]
    dx_star  = params["dx_star"]
    x0_star  = params["x0_star"]

    u_star = u_star_0.copy()
    v_star = np.zeros(N)
    u_star_prev = None                           # No previous step initially -> use 1st order du/dx in compute_v_star

    stride = max(1, Nx // 1000)                  # store solution every stride steps

    store_indices = set(range(0, Nx + 1, stride)) # indices/step numbers at which to store solution

    for x_out in params["x_output"]:
```

```python
        idx = int(round((x_out - x0_star) / dx_star)) # find step number corresponding to x_out
        idx = max(0, min(idx, Nx))                # ensure idx is within bounds
        store_indices.add(idx)                    # add idx to set store_indices (if not already present)

    store_indices = sorted(store_indices)         # sort indices for ordered storage
    x_vals = np.zeros(len(store_indices))         # initialize array to store solution x-values
    u_store = np.zeros((len(store_indices), N))   # initialize array to store solution profiles

    store_map = {idx: pos for pos, idx in enumerate(store_indices)} # map from step number to storage index

    if 0 in store_map:
        x_vals[store_map[0]] = x0_star            # store initial condition at x0_star if it's in the output list
        u_store[store_map[0], :] = u_star.copy()

    for n in range(Nx):
        nu_star, dnu_star_deta = compute_nu(params, u_star, J_star)
        A_j, B_j = compute_coefficients(params, v_star, J_star, J_star_eta, nu_star, dnu_star_deta)
        ab, rhs_vec, l_diag, u_diag = build_system(params, u_star, A_j, B_j)
        u_interior = solve_banded((l_diag, u_diag), ab, rhs_vec)

        u_star_new          = np.zeros(N)
        u_star_new[0]       = 0.0          # wall BC
        u_star_new[1:N-1]   = u_interior   # interior points
        u_star_new[N-1]     = 1.0          # freestream BC

        # Compute v_star using continuity equation
        v_star = compute_v_star(u_star_new, u_star, u_star_prev, params, J_star)

        u_star_prev = u_star.copy() # store current u_star before updating for next iteration
        u_star = u_star_new.copy() # update u_star for next iteration

        step = n + 1
        if step in store_map:
            pos = store_map[step]                # find storage index for current step
            x_vals[pos] = x0_star + step * dx_star # compute and store x-value for current step
            u_store[pos, :] = u_star.copy()      # store solution profile for current step

        # if step % stride == 0:
        if (n + 1) % 1000 == 0:
            # print(f" Step {step}/{Nx}, x* = {x0_star + (step) * dx_star:.2f}")
            print(f" Step {n+1}/{Nx}, x* = {x0_star + (n+1) * dx_star:.2f}")

    return x_vals, u_store

# ------------------------------------------------------------------
```

```python
# 9. COMPUTE BLASIUS SOLUTION
# ------------------------------------------------------------------

def compute_blasius(eta_max=10.0, n_points = 10000):
    """Solve Blasius via RK4."""

    h = eta_max / n_points       # step size for RK4

    eta_B = np.zeros(n_points + 1)      # values for eta
    f_arr = np.zeros(n_points + 1)      # stream function at each eta
    F_arr = np.zeros(n_points + 1)      # velocity profile u/U_inf = u*
    H_arr = np.zeros(n_points + 1)      # d2f/deta2

    f_arr[0] = 0.0                      # f(0) = 0 at the wall
    F_arr[0] = 0.0                      # F(0) = 0 at the wall
    H_arr[0] = 0.332                    # H(0) that results in F(n->infty) = 1

    def rhs(f_val, F_val, H_val):

        df = F_val                      # df/deta = F
        dF = H_val                      # dF/deta = H
        dH = -0.5 * f_val * H_val       # dH/deta = -(f/2)*H

        return df, dF, dH

    # Implement RK4 numerical method
    for i in range(n_points):
        eta_B[i] = i * h
        fi, Fi, Hi = f_arr[i], F_arr[i], H_arr[i]

        k1f, k1F, k1H = rhs(fi, Fi, Hi)
        k2f, k2F, k2H = rhs(fi + 0.5*h*k1f, Fi + 0.5*h*k1F, Hi + 0.5*h*k1H)
        k3f, k3F, k3H = rhs(fi + 0.5*h*k2f, Fi + 0.5*h*k2F, Hi + 0.5*h*k2H)
        k4f, k4F, k4H = rhs(fi + h*k3f, Fi + h*k3F, Hi + h*k3H)

        f_arr[i+1] = fi + (h/6.0)*(k1f + 2*k2f + 2*k3f + k4f)
        F_arr[i+1] = Fi + (h/6.0)*(k1F + 2*k2F + 2*k3F + k4F)
        H_arr[i+1] = Hi + (h/6.0)*(k1H + 2*k2H + 2*k3H + k4H)

    eta_B[-1] = n_points * h

    return eta_B, F_arr, f_arr # eta_B, u*/U_infty = F, stream function f

# ------------------------------------------------------------------
# 10. COMPUTE THICKNESSES AND SHAPE FACTOR
# ------------------------------------------------------------------

def compute_thicknesses(u_star, J_star, deta):
    """Compute displacement thickness, momentum thickness, and shape factor"""
    integrand_disp  = (1.0 - u_star) * J_star
    integrand_mom   = u_star * (1.0 - u_star) * J_star

    delta_star      = np.trapezoid(integrand_disp, dx=deta)  #integrate for displacement
        thickness using trapezoidal rule
```

```python
    theta           = np.trapezoid(integrand_mom, dx=deta)   #integrate for momentum thickness using trapezoidal rule

    H               = delta_star / theta if theta > 1e-15 else 0.0

    return delta_star, theta, H

# ------------------------------------------------------------------
# 11. POST-PROCESSING
# ------------------------------------------------------------------

def post_process(params, x_vals, u_store, eta, y_phys, y_star, J_star):
    N         = params["N"]
    deta      = params["deta"]
    nu        = params["nu_inf"]
    U_inf     = params["U_inf"]
    L         = params["L"]
    delta     = params["delta"]

    print("Computing Blasius solution...")
    eta_B, F_B, f_B = compute_blasius(eta_max=10.0, n_points = 10000)

    output_indices = []
    for x_out in params["x_output"]:
        idx = np.argmin(np.abs(x_vals - x_out))     #take difference between desired x and stored x and return index of lowest entry
        output_indices.append(idx)

    colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red', 'tab:purple']

    # Plot 01: u* vs. y/delta
    #plt.figure(figsize=(8,6))
    plt.figure(figsize=(6,5))
    for i, idx in enumerate(output_indices):
        x_now = x_vals[idx]; x_dim = x_now * L
        u_num = u_store[idx, :]

        plt.plot(u_num, y_star, '-', color=colors[i], linewidth=1.5, label = fr"Numerical, $x^*={x_now:.2f}$")

        if i > 0:
            scale = np.sqrt(U_inf / (nu * x_dim))     # compute scaling factor for Blasius Similarity Variable
            eta_B_local = y_phys * scale              # compute Blasius Similarity Variable for desired locations

            u_blasius = np.interp(eta_B_local, eta_B, F_B, right=1.0) # interpolate for Blasius solution values using eta_b_local array

            plt.plot(u_blasius, y_star, '--', color=colors[i], linewidth=1.5, label = fr"Blasius, $x^*={x_now:.2f}$")

    plt.xlabel(fr'$u^* = u / U_\infty$', fontsize=14)
    plt.ylabel(fr'$y^* = y / \delta$', fontsize =14)
```

```python
    plt.xlim([-0.05, 1.1]); plt.ylim([0, 4])
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(fontsize=14, loc='best')
    plt.tight_layout()
    plt.savefig('velocity_profiles.png', dpi=300, bbox_inches='tight')
    print("...Saved velocity_profiles.png")

    #Plot 02: Similarity Profiles
    #plt.figure(figsize=(8,6))
    plt.figure(figsize=(6,5))
    for i, idx in enumerate(output_indices):
        x_now = x_vals[idx]; x_dim = x_now * L
        u_num = u_store[idx, :]

        scale = np.sqrt(U_inf / (nu * x_dim))
        eta_B_local = y_phys * scale

        plt.plot(u_num, eta_B_local, '-', color=colors[i], linewidth=1.5, label=fr"$x^*={x_now:.2f}$")
    plt.plot(F_B, eta_B, 'k--', linewidth=2, label='Blasius')
    plt.xlabel(fr'$u^* = u / U_\infty$', fontsize=14)
    plt.ylabel(fr'$\eta_B$', fontsize =14)
    plt.xlim([-0.05, 1.1]); plt.ylim([0, 8])
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(fontsize=14, loc='best')
    plt.tight_layout()
    plt.savefig('similarity_profiles.png', dpi=300, bbox_inches='tight')
    print("...Saved similarity_profiles.png")

    #Plot 03: L2 Residual
    plt.figure(figsize=(6,5))
    residuals = []; x_res = []
    for i in range(1, len(x_vals)):      # start from 1 so we can compare to previous step
        dx_gap = x_vals[i] - x_vals[i-1] # take difference between stored x-values
        if dx_gap < 1e-15: continue      # skip loop if comparing data from same x-values

        res = np.linalg.norm(u_store[i, :] - u_store[i-1,:], ord=2) / dx_gap # normalize to step size in x between stored solutions
        residuals.append(res); x_res.append(x_vals[i])

    plt.plot(x_res, residuals, 'o-', color='tab:blue', linewidth=1.5, markersize=2)
    plt.xlabel(r'$x^*$', fontsize=14)
    plt.ylabel(r'$L_2$ Residual: $\|u^{n+1} - u^n\|_2 / \Delta x^*$', fontsize=14)
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5)
    #plt.legend(fontsize=10, loc='best')
    plt.tight_layout()
    plt.savefig('residual_vs_x.png', dpi=300, bbox_inches='tight')
    print("...Saved residual_vs_x.png")

    #Plot 04: RMS Error
```

```python
    plt.figure(figsize=(6,5))
    errors = []; x_errors = []; rms_vals = []
    for i in range(1, len(x_vals)):
        x_now = x_vals[i]; x_dim = x_now * L
        u_num = u_store[i, :]

        if x_dim == 0: continue

        scale = np.sqrt(U_inf / (nu * x_dim))     # compute scaling factor for Blasius Similarity Variable
        eta_B_local = y_phys * scale              # compute Blasius Similarity Variable for desired locations

        u_blasius = np.interp(eta_B_local, eta_B, F_B, right=1.0) # interpolate for Blasius solution values using eta_b_local array

        err = np.linalg.norm(u_num - u_blasius, ord=2)
        rms = err / np.sqrt(len(u_num))

        errors.append(err); x_errors.append(x_now); rms_vals.append(rms)

    plt.plot(x_errors, rms_vals, 'o-', color='tab:red', linewidth=1.5, markersize=2)
    #plt.ylabel(r'L2 Error: $\|u_{num} - u_{Blas}\|_2$', fontsize=18)
    plt.ylabel(r'RMS Error', fontsize=14)
    plt.xlabel(fr'$x^* = x / L$', fontsize =14)
    #plt.xlim([-0.05, 1.1]); plt.ylim([0, 4])
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5)
    #plt.legend(fontsize=9, loc='best')
    plt.tight_layout()
    plt.savefig('error_vs_x.png', dpi=300, bbox_inches='tight')
    print("...Saved error_vs_x.png")

    #Plot 05: Boundary Layer Growth
    plt.figure(figsize=(8,6))
    for i, idx in enumerate(output_indices):
        x_now = x_vals[idx]; u_num = u_store[idx, :]
        plt.plot(y_star, u_num, '-', color=colors[i], linewidth=1.5, label=fr"$x^*={x_now:.0f}$")
    plt.ylabel(fr'$u^* = u / U_\infty$', fontsize=14)
    plt.xlabel(fr'$y^* = y / \delta$', fontsize =14)
    plt.xlim([0,6])
    plt.xticks(fontsize=12); plt.yticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(fontsize=10, loc='best')
    plt.tight_layout()
    plt.savefig('boundary_layer_growth.png', dpi=300, bbox_inches='tight')
    print("...Saved boundary_layer_growth.png")

    #Table: Thicknesses
    print("\n" + "="*120)
    print(f"{'x*':>10} {'x [m]':>10} {'d*/d':>12} {'th/d':>12} "
          f"{'H':>12} {'d*_Blas/d':>15} {'th_Blas/d':>15} {'H_Blas':>12}")
    print("="*120)
```

```python
    thickness_data = []
    for i, idx in enumerate(output_indices):
        x_now = x_vals[idx]; x_dim = x_now * L
        u_num = u_store[idx, :]
        delta_star, theta, H = compute_thicknesses(u_num, J_star, deta)
        sqrt_nu_x_U = np.sqrt(nu * x_dim / U_inf)
        delta_star_blas = 1.72 * sqrt_nu_x_U / delta
        theta_blas     = 0.664 * sqrt_nu_x_U / delta
        H_blas = delta_star_blas / theta_blas if theta_blas > 0 else 0.0
        thickness_data.append((x_now, x_dim, delta_star, theta, H, delta_star_blas, theta_blas, H_blas))
        print(f"{x_now:10.2f} {x_dim:10.2f} {delta_star:12.6f} {theta:12.6f} {H:12.4f} {delta_star_blas:15.6} {theta_blas:15.6} {H_blas:12.4f}")
    print("="*120 )

    #Print x0 calculation summary
    print(f"\nInitial x-location calculation:")
    print(f"    From Blasius: delta = c*sqrt(x0), c = 4.91*sqrt(nu/U)")
    print(f"    c = 4.91*sqrt({nu}/{U_inf}) = {4.91*np.sqrt(nu/U_inf):.6e}")
    print(f"    x0 = delta^2/c^2 = ({delta})^2 / ({4.91*np.sqrt(nu/U_inf):.6e})^2 = {params['x0']:.2f} m")
    print(f"    xo* = x0/L = {params['x0_star']:.2f}")
    print(f"    Re_x0 = U_inf*x0/nu = {U_inf*params['x0']/nu:.2e}")

    return thickness_data

# ------------------------------------------------------------------
# 12. MAIN DRIVER
# ------------------------------------------------------------------
def main():
    print("=" * 120)
    print( "    ME 5311 - Project 02: Laminar Boundary Layer Solver")
    print("=" * 120)

    print("\n[1] Setting parameters...")
    params = set_parameters()
    print(f"    Re_L = {params['Re_L']:.2e}")
    print(f"    x0*  = {params['x0_star']:.2f}")

    print("\n[2] Computing stretched grid...")
    eta, y_phys, y_star, J_star, J_star_eta = compute_grid(params)
    print(f"    dy_min = {y_phys[1]-y_phys[0]:.2e} m, dy_max = {y_phys[-1]-y_phys[-2]:.2e} m")

    print("\n[3] Setting initial conditions...")
    u_star_0 = compute_initial_condition(params, y_star)

    print("\n[4] Marching in x...")
    x_vals, u_store = step_in_x(params, u_star_0, J_star, J_star_eta)

    print("\n[5] Post-processing...")
    post_process(params, x_vals, u_store, eta, y_phys, y_star, J_star)

    print("\nDone!")
```

```python

if __name__ == "__main__":
    main()
```